EXPERIMENT-3

Name: Pranav Narayanan.T Roll No:240701393

1.COMMAND LINE INTERFACE:

PYTHON CODE:

```
"""

Smart Task Manager (CLI) - Professional Command Line Interface
Application

A text-based task management system for terminal usage.

Part of a college UI/HCI lab project demonstrating CLI, GUI, and VUI
interfaces.

"""



import os

import sys




#
==========================================================================
======

# GLOBAL DATA STORE

#
==========================================================================
======

# In-memory storage for tasks

tasks = []

task_id_counter = 0




#
==========================================================================
```

```
======

# UTILITY FUNCTIONS

#
=========================================================================
======



def clear_screen():

    """Clear the terminal screen for better readability."""

    os.system('cls' if os.name == 'nt' else 'clear')




def print_header():

    """Print the application header."""

    print("=" * 60)

    print("              SMART TASK MANAGER (CLI)")

    print("         Organize your work efficiently")

    print("=" * 60)

    print()




def print_separator():

    """Print a separator line."""

    print("-" * 60)




def print_menu():

    """Display the main menu options."""
```

```
    print("\n[ MAIN MENU ]")

    print("1. Add a new task")

    print("2. View all tasks")

    print("3. Mark a task as completed")

    print("4. Delete a task")

    print("5. Exit")

    print_separator()




def get_valid_menu_choice():

    """Get and validate menu choice from user."""

    while True:

        try:

            choice = int(input("Enter your choice (1-5): ").strip())

            if 1 <= choice <= 5:

                return choice

            else:

                print("Error: Please enter a number between 1 and 5.")

        except ValueError:

            print("Error: Invalid input. Please enter a number.")




def get_valid_task_number():

    """Get and validate task number from user."""

    while True:

        try:
```

```
            task_num = int(input("Enter task number: ").strip())

            if task_num > 0 and task_num <= len(tasks):

                return task_num

            else:

                print(f"Error: Please enter a valid task number
(1-{len(tasks)}).")

        except ValueError:

            print("Error: Invalid input. Please enter a number.")




def pause():

    """Pause to let user read the output."""

    input("\nPress Enter to continue...")




#
==========================================================================
======

# TASK OPERATIONS

#
==========================================================================
======



def add_task():

    """Add a new task to the task list."""

    print("\n[ ADD NEW TASK ]")

    print_separator()
```

```
    task_description = input("Enter task description: ").strip()


    # Validate input
    if not task_description:
        print("\nError: Task description cannot be empty.")
        return


    # Create new task
    global task_id_counter
    task_id_counter += 1


    new_task = {
        'id': task_id_counter,
        'description': task_description,
        'status': 'Pending'
    }


    tasks.append(new_task)


    print(f"\nSuccess: Task '{task_description}' added successfully!")
    print(f"         Task #{len(tasks)} has been created.")



def view_tasks():
    """Display all tasks in a formatted view."""
    print("\n[ VIEW ALL TASKS ]")
```

```
    print_separator()


    if not tasks:
        print("\nNo tasks found. Add some tasks to get started!")
        return


    # Print table header
    print(f"\n{'#':<5} {'Task Description':<40} {'Status':<15}")
    print_separator()


    # Print each task
    for index, task in enumerate(tasks, start=1):
        status_display = task['status']


        # Color coding using ANSI codes (works in most terminals)
        if task['status'] == 'Completed':
            status_display = f"\033[92m{task['status']}\033[0m"  #
Green
        else:
            status_display = f"\033[90m{task['status']}\033[0m"  # Gray


        # Truncate long descriptions
        desc = task['description'][:38] + ".." if
len(task['description']) > 40 else task['description']


        print(f"{index:<5} {desc:<40} {status_display:<15}")
```

```
    print_separator()

    print(f"\nTotal Tasks: {len(tasks)}")

    completed_count = sum(1 for t in tasks if t['status'] ==
'Completed')

    pending_count = len(tasks) - completed_count

    print(f"Completed: {completed_count} | Pending: {pending_count}")




def mark_task_complete():

    """Mark a selected task as completed."""

    print("\n[ MARK TASK AS COMPLETED ]")

    print_separator()


    if not tasks:

        print("\nError: No tasks available. Add some tasks first.")

        return


    # Show current tasks

    print("\nCurrent Tasks:")

    for index, task in enumerate(tasks, start=1):

        status_marker = "✓" if task['status'] == 'Completed' else " "

        print(f"  {index}. [{status_marker}] {task['description']}")


    print()

    task_num = get_valid_task_number()
```

```
    # Get the task
    task = tasks[task_num - 1]

    # Check if already completed
    if task['status'] == 'Completed':
        print(f"\nError: Task #{task_num} is already completed!")
        return

    # Mark as completed
    task['status'] = 'Completed'
    print(f"\nSuccess: Task #{task_num} marked as completed! ✓")


def delete_task():
    """Delete a selected task."""
    print("\n[ DELETE TASK ]")
    print_separator()

    if not tasks:
        print("\nError: No tasks available. Add some tasks first.")
        return

    # Show current tasks
    print("\nCurrent Tasks:")
    for index, task in enumerate(tasks, start=1):
        status_marker = "✓" if task['status'] == 'Completed' else " "
```

```
        print(f"  {index}. [{status_marker}] {task['description']}")



    print()

    task_num = get_valid_task_number()



    # Get the task to delete

    task_to_delete = tasks[task_num - 1]



    # Confirm deletion

    print(f"\nAre you sure you want to delete:
'{task_to_delete['description']}'?")

    confirm = input("Type 'yes' to confirm: ").strip().lower()



    if confirm == 'yes':

        deleted_task = tasks.pop(task_num - 1)

        print(f"\nSuccess: Task '{deleted_task['description']}' deleted
successfully!")

    else:

        print("\nDeletion cancelled.")





#
==========================================================================
======

# MAIN APPLICATION LOOP

#
==========================================================================
======
```

```python
def main():
    """Main function to run the CLI application."""
    # Main application loop
    while True:
        clear_screen()
        print_header()
        print_menu()

        # Get user choice
        choice = get_valid_menu_choice()

        # Process user choice
        if choice == 1:
            add_task()
            pause()
        elif choice == 2:
            view_tasks()
            pause()
        elif choice == 3:
            mark_task_complete()
            pause()
        elif choice == 4:
            delete_task()
            pause()
        elif choice == 5:
            clear_screen()
```

```
            print_header()

            print("\nThank you for using Smart Task Manager!")

            print("Goodbye!\n")

            break




#
==========================================================================
======

# ENTRY POINT

#
==========================================================================
======



if __name__ == "__main__":

    main()
```

OUTPUT:

```
PROBLEMS 2    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

              SMART TASK MANAGER (CLI)
          Organize your work efficiently
  ============================================================


  [ MAIN MENU ]
  1. Add a new task
  2. View all tasks
  3. Mark a task as completed
  4. Delete a task
  5. Exit
  ------------------------------------------------------------
  Enter your choice (1-5):
```

2.GRAPHICAL USER INTERFACE:

PYTHON CODE:

```
"""
Smart Task Manager - Professional Task Management Application
A modern, production-style Tkinter application for managing
```

```
tasks.
Designed for college UI/HCI lab usage.
"""

import tkinter as tk
from tkinter import ttk, messagebox
import sys


#
================================================================
================
# COLOR SCHEME - Dark Theme Configuration
#
================================================================
================
COLORS = {
    'bg_primary': '#1E1E2E',        # Dark charcoal background
    'bg_secondary': '#2D2D44',       # Slightly lighter for
cards
    'bg_tertiary': '#3D3D5C',        # For hover states
    'accent': '#7C3AED',             # Purple accent color
    'accent_hover': '#8B5CF6',       # Lighter purple for
hover
    'success': '#10B981',            # Green for completed
    'success_bg': '#064E3B',         # Dark green background
    'pending': '#6B7280',            # Gray for pending
    'pending_bg': '#374151',         # Dark gray background
    'text_primary': '#FFFFFF',       # White text
    'text_secondary': '#A0AEC0',     # Light gray text
    'text_muted': '#718096',         # Muted text
    'border': '#4A4A6A',             # Border color
    'error': '#EF4444',              # Red for errors
    'sidebar': '#16162A',            # Sidebar background
}


#
================================================================
================
# FONTS - Typography Configuration
#
================================================================
================
FONTS = {
    'title': ('Segoe UI', 28, 'bold'),
    'subtitle': ('Segoe UI', 12),
```

```
    'heading': ('Segoe UI', 16, 'bold'),
    'body': ('Segoe UI', 11),
    'body_bold': ('Segoe UI', 11, 'bold'),
    'button': ('Segoe UI', 11, 'bold'),
    'small': ('Segoe UI', 9),
}


class SmartTaskManager:
    """Main application class for Smart Task Manager."""

    def __init__(self, root):
        """Initialize the application with dark theme
styling."""
        self.root = root
        self.root.title("Smart Task Manager")
        self.root.geometry("900x550")
        self.root.resizable(False, False)

        # Center the window on screen
        self.center_window()

        # Set dark theme background
        self.root.configure(bg=COLORS['bg_primary'])

        # Task storage (in-memory)
        self.tasks = []
        self.task_id_counter = 0

        # Configure ttk styles for modern look
        self.setup_styles()

        # Build the UI
        self.create_ui()

        # Bind events
        self.root.protocol("WM_DELETE_WINDOW",
self.on_closing)

    def center_window(self):
        """Center the window on the screen."""
        self.root.update_idletasks()
        width = 900
        height = 550
        x = (self.root.winfo_screenwidth() // 2) - (width //
2)
        y = (self.root.winfo_screenheight() // 2) - (height
```

```
// 2)
        self.root.geometry(f'{width}x{height}+{x}+{y}')

    def setup_styles(self):
        """Configure ttk styles for modern appearance."""
        style = ttk.Style()
        style.theme_use('clam')

        # Configure Treeview (task list) style
        style.configure(
            'Modern.Treeview',
            background=COLORS['bg_secondary'],
            foreground=COLORS['text_primary'],
            fieldbackground=COLORS['bg_secondary'],
            borderwidth=0,
            font=FONTS['body'],
            rowheight=45,
        )
        style.configure(
            'Modern.Treeview.Heading',
            background=COLORS['sidebar'],
            foreground=COLORS['text_primary'],
            font=FONTS['body_bold'],
            borderwidth=0,
        )
        style.map(
            'Modern.Treeview',
            background=[('selected', COLORS['accent'])],
        )

        # Configure Scrollbar style
        style.configure(
            'Modern.Vertical.TScrollbar',
            background=COLORS['bg_secondary'],
            troughcolor=COLORS['bg_primary'],
            borderwidth=0,
        )

    def create_ui(self):
        """Create the complete user interface."""
        # ==================== MAIN CONTAINER
====================
        main_container = tk.Frame(
            self.root,
            bg=COLORS['bg_primary']
        )
        main_container.pack(fill=tk.BOTH, expand=True)
```

```
        # ==================== SIDEBAR ====================
        self.create_sidebar(main_container)

        # ==================== MAIN CONTENT AREA
====================
        self.create_main_content(main_container)

    def create_sidebar(self, parent):
        """Create the sidebar with title and navigation."""
        sidebar = tk.Frame(
            parent,
            bg=COLORS['sidebar'],
            width=250
        )
        sidebar.pack(side=tk.LEFT, fill=tk.Y)
        sidebar.pack_propagate(False)

        # App Logo/Icon Area
        logo_frame = tk.Frame(sidebar, bg=COLORS['sidebar'],
height=100)
        logo_frame.pack(fill=tk.X, pady=(30, 10))

        # App Icon (circle with checkmark)
        icon_canvas = tk.Canvas(
            logo_frame,
            width=60,
            height=60,
            bg=COLORS['sidebar'],
            highlightthickness=0
        )
        icon_canvas.pack()
        icon_canvas.create_oval(5, 5, 55, 55,
fill=COLORS['accent'], outline='')
        icon_canvas.create_text(30, 30, text='✓',
font=('Segoe UI', 24, 'bold'),
                                fill='white',
tags='checkmark')

        # App Title
        title_label = tk.Label(
            sidebar,
            text="Smart Task\nManager",
            font=FONTS['title'],
            bg=COLORS['sidebar'],
            fg=COLORS['text_primary'],
            justify=tk.CENTER
```

```
        )
        title_label.pack(pady=(10, 5))

        # Subtitle
        subtitle_label = tk.Label(
            sidebar,
            text="Organize your work efficiently",
            font=FONTS['subtitle'],
            bg=COLORS['sidebar'],
            fg=COLORS['text_secondary']
        )
        subtitle_label.pack(pady=(0, 30))

        # Stats Section
        self.create_stats_section(sidebar)

        # Spacer
        tk.Frame(sidebar,
bg=COLORS['sidebar']).pack(fill=tk.Y, expand=True)

        # Footer
        footer_label = tk.Label(
            sidebar,
            text="v1.0 | College UI Lab",
            font=FONTS['small'],
            bg=COLORS['sidebar'],
            fg=COLORS['text_muted']
        )
        footer_label.pack(pady=(0, 20))

    def create_stats_section(self, parent):
        """Create statistics display in sidebar."""
        stats_frame = tk.Frame(
            parent,
            bg=COLORS['bg_secondary'],
            padx=20,
            pady=20
        )
        stats_frame.pack(fill=tk.X, padx=15, pady=(0, 20))

        # Stats Title
        stats_title = tk.Label(
            stats_frame,
            text="STATISTICS",
            font=FONTS['small'],
            bg=COLORS['bg_secondary'],
            fg=COLORS['text_muted']
```

```
        )
        stats_title.pack(anchor=tk.W, pady=(0, 15))

        # Total Tasks
        total_frame = tk.Frame(stats_frame,
bg=COLORS['bg_secondary'])
        total_frame.pack(fill=tk.X, pady=5)
        tk.Label(
            total_frame,
            text="Total Tasks:",
            font=FONTS['body'],
            bg=COLORS['bg_secondary'],
            fg=COLORS['text_secondary']
        ).pack(side=tk.LEFT)
        self.total_label = tk.Label(
            total_frame,
            text="0",
            font=FONTS['body_bold'],
            bg=COLORS['bg_secondary'],
            fg=COLORS['text_primary']
        )
        self.total_label.pack(side=tk.RIGHT)

        # Completed Tasks
        completed_frame = tk.Frame(stats_frame,
bg=COLORS['bg_secondary'])
        completed_frame.pack(fill=tk.X, pady=5)
        tk.Label(
            completed_frame,
            text="Completed:",
            font=FONTS['body'],
            bg=COLORS['bg_secondary'],
            fg=COLORS['text_secondary']
        ).pack(side=tk.LEFT)
        self.completed_label = tk.Label(
            completed_frame,
            text="0",
            font=FONTS['body_bold'],
            bg=COLORS['bg_secondary'],
            fg=COLORS['success']
        )
        self.completed_label.pack(side=tk.RIGHT)

        # Pending Tasks
        pending_frame = tk.Frame(stats_frame,
bg=COLORS['bg_secondary'])
        pending_frame.pack(fill=tk.X, pady=5)
```

```
        tk.Label(
            pending_frame,
            text="Pending:",
            font=FONTS['body'],
            bg=COLORS['bg_secondary'],
            fg=COLORS['text_secondary']
        ).pack(side=tk.LEFT)
        self.pending_label = tk.Label(
            pending_frame,
            text="0",
            font=FONTS['body_bold'],
            bg=COLORS['bg_secondary'],
            fg=COLORS['pending']
        )
        self.pending_label.pack(side=tk.RIGHT)

    def create_main_content(self, parent):
        """Create the main content area with task list and
controls."""
        # Main content frame
        content = tk.Frame(
            parent,
            bg=COLORS['bg_primary']
        )
        content.pack(side=tk.LEFT, fill=tk.BOTH, expand=True,
padx=30, pady=30)

        # ==================== HEADER SECTION
====================
        header = tk.Frame(content, bg=COLORS['bg_primary'])
        header.pack(fill=tk.X, pady=(0, 20))

        # Section Title
        section_title = tk.Label(
            header,
            text="My Tasks",
            font=FONTS['heading'],
            bg=COLORS['bg_primary'],
            fg=COLORS['text_primary']
        )
        section_title.pack(side=tk.LEFT)

        # ==================== INPUT SECTION
====================
        input_frame = tk.Frame(content,
bg=COLORS['bg_primary'])
        input_frame.pack(fill=tk.X, pady=(0, 20))
```

```
        # Task Input Field
        self.task_entry = tk.Entry(
            input_frame,
            font=FONTS['body'],
            bg=COLORS['bg_secondary'],
            fg=COLORS['text_primary'],
            insertbackground=COLORS['text_primary'],
            relief=tk.FLAT,
            bd=0,
            highlightthickness=1,
            highlightcolor=COLORS['accent'],
            highlightbackground=COLORS['border']
        )
        self.task_entry.pack(side=tk.LEFT, fill=tk.X,
expand=True, ipady=12, ipadx=15)
        self.task_entry.insert(0, "Enter a new task...")
        self.task_entry.bind('<FocusIn>', lambda e:
self.on_entry_focus_in())
        self.task_entry.bind('<FocusOut>', lambda e:
self.on_entry_focus_out())

        # Add Button
        add_button = tk.Button(
            input_frame,
            text="+ Add Task",
            font=FONTS['button'],
            bg=COLORS['accent'],
            fg=COLORS['text_primary'],
            activebackground=COLORS['accent_hover'],
            activeforeground=COLORS['text_primary'],
            relief=tk.FLAT,
            bd=0,
            cursor='hand2',
            padx=20,
            pady=12,
            command=self.add_task
        )
        add_button.pack(side=tk.LEFT, padx=(15, 0))

        # Bind Enter key to add task
        self.task_entry.bind('<Return>', lambda e:
self.add_task())

        # ==================== TASK LIST SECTION
====================
        list_frame = tk.Frame(content,
```

```
bg=COLORS['bg_primary'])
        list_frame.pack(fill=tk.BOTH, expand=True)

        # Treeview for tasks
        columns = ('id', 'task', 'status')
        self.tree = ttk.Treeview(
            list_frame,
            columns=columns,
            show='tree headings',
            style='Modern.Treeview',
            selectmode='browse'
        )

        # Configure columns
        self.tree.heading('#0', text='')
        self.tree.heading('id', text='ID')
        self.tree.heading('task', text='Task')
        self.tree.heading('status', text='Status')

        self.tree.column('#0', width=0, stretch=tk.NO)
        self.tree.column('id', width=60, minwidth=60,
anchor=tk.CENTER)
        self.tree.column('task', width=400, minwidth=300)
        self.tree.column('status', width=120, minwidth=100,
anchor=tk.CENTER)

        # Scrollbar
        scrollbar = ttk.Scrollbar(
            list_frame,
            orient=tk.VERTICAL,
            command=self.tree.yview,
            style='Modern.Vertical.TScrollbar'
        )
        self.tree.configure(yscrollcommand=scrollbar.set)

        # Pack tree and scrollbar
        self.tree.pack(side=tk.LEFT, fill=tk.BOTH,
expand=True)
        scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

        # Bind selection event
        self.tree.bind('<<TreeviewSelect>>',
self.on_task_select)

        # ==================== ACTION BUTTONS SECTION
====================
        actions_frame = tk.Frame(content,
```

```
bg=COLORS['bg_primary'])
        actions_frame.pack(fill=tk.X, pady=(20, 0))

        # Complete Task Button
        complete_btn = tk.Button(
            actions_frame,
            text="✓ Mark Complete",
            font=FONTS['button'],
            bg=COLORS['success'],
            fg=COLORS['text_primary'],
            activebackground=COLORS['success_bg'],
            activeforeground=COLORS['text_primary'],
            relief=tk.FLAT,
            bd=0,
            cursor='hand2',
            padx=25,
            pady=12,
            command=self.mark_complete
        )
        complete_btn.pack(side=tk.LEFT, padx=(0, 10))

        # Delete Task Button
        delete_btn = tk.Button(
            actions_frame,
            text="✗ Delete Task",
            font=FONTS['button'],
            bg=COLORS['error'],
            fg=COLORS['text_primary'],
            activebackground='#B91C1C',
            activeforeground=COLORS['text_primary'],
            relief=tk.FLAT,
            bd=0,
            cursor='hand2',
            padx=25,
            pady=12,
            command=self.delete_task
        )
        delete_btn.pack(side=tk.LEFT)

        # Selected task label
        self.selected_label = tk.Label(
            actions_frame,
            text="No task selected",
            font=FONTS['small'],
            bg=COLORS['bg_primary'],
            fg=COLORS['text_muted']
        )
```

```
        self.selected_label.pack(side=tk.RIGHT, pady=(0, 0))

    # ==================== EVENT HANDLERS
====================

    def on_entry_focus_in(self):
        """Handle entry focus in event."""
        if self.task_entry.get() == "Enter a new task...":
            self.task_entry.delete(0, tk.END)
            self.task_entry.configure(
                fg=COLORS['text_primary'],
                highlightcolor=COLORS['accent'],
                highlightbackground=COLORS['border']
            )

    def on_entry_focus_out(self):
        """Handle entry focus out event."""
        if not self.task_entry.get():
            self.task_entry.insert(0, "Enter a new task...")

self.task_entry.configure(fg=COLORS['text_muted'])

    def on_task_select(self, event):
        """Handle task selection event."""
        selection = self.tree.selection()
        if selection:
            item = self.tree.item(selection[0])
            task_id = item['values'][0]
            task_text = item['values'][1]
            self.selected_label.config(text=f"Selected:
{task_text[:30]}...")
        else:
            self.selected_label.config(text="No task
selected")

    def on_closing(self):
        """Handle window closing event."""
        if messagebox.askokcancel("Exit", "Are you sure you
want to exit?"):
            self.root.destroy()
            sys.exit()

    # ==================== TASK OPERATIONS
====================

    def add_task(self):
        """Add a new task to the list."""
```

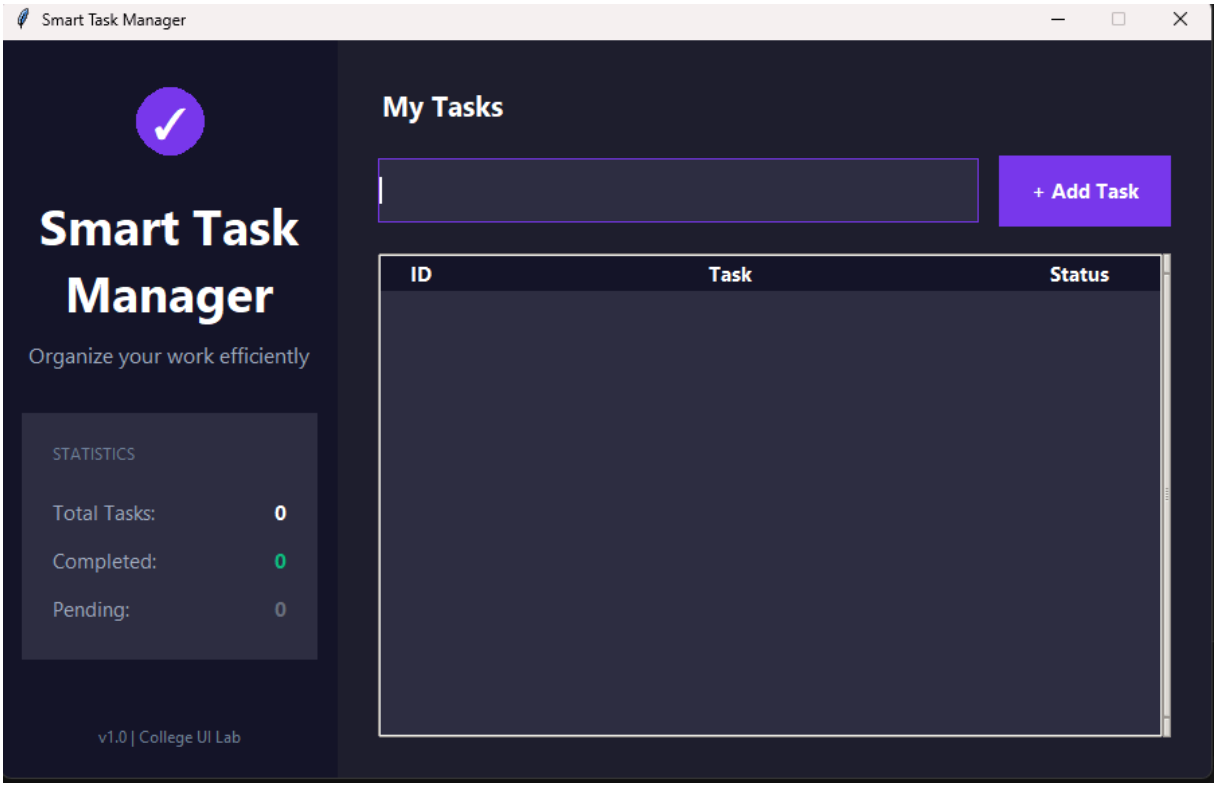
```
        task_text = self.task_entry.get().strip()

        # Input validation
        if not task_text or task_text == "Enter a new
task...":
            self.show_error("Please enter a task
description.")
            self.task_entry.focus()
            return

        # Create task
        self.task_id_counter += 1
        task = {
            'id': self.task_id_counter,
            'text': task_text,
            'status': 'Pending'
        }
        self.tasks.append(task)

        # Insert into treeview
        self.tree.insert(
            '',
            'end',
            values=(task['id'], task['text'],
task['status']),
            tags=('pending',)
        )

        # Clear entry
        self.task_entry.delete(0, tk.END)
        self.task_entry.insert(0, "Enter a new task...")
        self.task_entry.configure(fg=COLORS['text_muted'])

        # Update stats
        self.update_stats()

        # Show success message
        self.show_success("Task added successfully!")

        # Select the newly added task
        last_item = self.tree.get_children()[-1]
        self.tree.selection_set(last_item)
        self.tree.focus(last_item)

    def mark_complete(self):
        """Mark selected task as completed."""
        selection = self.tree.selection()
```

```
        if not selection:
            self.show_error("Please select a task to mark as
complete.")
            return

        item = selection[0]
        item_values = self.tree.item(item)['values']

        # Check if already completed
        if item_values[2] == 'Completed':
            self.show_error("This task is already
completed!")
            return

        # Update task in memory
        task_id = item_values[0]
        for task in self.tasks:
            if task['id'] == task_id:
                task['status'] = 'Completed'
                break

        # Update treeview
        self.tree.item(item, values=(item_values[0],
item_values[1], 'Completed'))

        # Update stats
        self.update_stats()

        # Show success
        self.show_success("Task marked as completed! ✓")

    def delete_task(self):
        """Delete selected task."""
        selection = self.tree.selection()
        if not selection:
            self.show_error("Please select a task to
delete.")
            return

        item = selection[0]
        item_values = self.tree.item(item)['values']

        # Confirm deletion
        if not messagebox.askyesno("Confirm Delete", f"Delete
task: '{item_values[1]}'?"):
            return
```

```
        # Remove from memory
        task_id = item_values[0]
        self.tasks = [task for task in self.tasks if
task['id'] != task_id]

        # Remove from treeview
        self.tree.delete(item)

        # Update stats
        self.update_stats()

        # Update selection label
        self.selected_label.config(text="No task selected")

        # Show success
        self.show_success("Task deleted successfully!")

    def update_stats(self):
        """Update the statistics display."""
        total = len(self.tasks)
        completed = sum(1 for task in self.tasks if
task['status'] == 'Completed')
        pending = total - completed

        self.total_label.config(text=str(total))
        self.completed_label.config(text=str(completed))
        self.pending_label.config(text=str(pending))

    def show_success(self, message):
        """Show success message popup."""
        messagebox.showinfo("Success", message)

    def show_error(self, message):
        """Show error message popup."""
        messagebox.showerror("Error", message)


#
================================================================
================
# MAIN ENTRY POINT
#
================================================================
================
def main():
    """Main function to run the application."""
    root = tk.Tk()
```

```
    app = SmartTaskManager(root)
    root.mainloop()


if __name__ == "__main__":
    main()
```

OUTPUT:



3.VOICE USER INTERFACE:

PYTHON CODE:

```
"""
Smart Task Manager - Professional Task Management Application
A modern, production-style Tkinter application for managing tasks.
Designed for college UI/HCI lab usage.
"""

import tkinter as tk
from tkinter import ttk, messagebox
import sys


#
==========================================================================
======
# COLOR SCHEME - Dark Theme Configuration
#
==========================================================================
======
COLORS = {
    'bg_primary': '#1E1E2E',        # Dark charcoal background
    'bg_secondary': '#2D2D44',       # Slightly lighter for cards
    'bg_tertiary': '#3D3D5C',        # For hover states
    'accent': '#7C3AED',             # Purple accent color
```

```
    'accent_hover': '#8B5CF6',      # Lighter purple for hover
    'success': '#10B981',           # Green for completed
    'success_bg': '#064E3B',        # Dark green background
    'pending': '#6B7280',           # Gray for pending
    'pending_bg': '#374151',        # Dark gray background
    'text_primary': '#FFFFFF',      # White text
    'text_secondary': '#A0AEC0',    # Light gray text
    'text_muted': '#718096',        # Muted text
    'border': '#4A4A6A',            # Border color
    'error': '#EF4444',             # Red for errors
    'sidebar': '#16162A',           # Sidebar background
}


#
==========================================================================
======
# FONTS - Typography Configuration
#
==========================================================================
======
FONTS = {
    'title': ('Segoe UI', 28, 'bold'),
    'subtitle': ('Segoe UI', 12),
    'heading': ('Segoe UI', 16, 'bold'),
    'body': ('Segoe UI', 11),
    'body_bold': ('Segoe UI', 11, 'bold'),
    'button': ('Segoe UI', 11, 'bold'),
    'small': ('Segoe UI', 9),
}


class SmartTaskManager:
    """Main application class for Smart Task Manager."""

    def __init__(self, root):
        """Initialize the application with dark theme styling."""
        self.root = root
        self.root.title("Smart Task Manager")
        self.root.geometry("900x550")
        self.root.resizable(False, False)

        # Center the window on screen
        self.center_window()
```

```python
        # Set dark theme background
        self.root.configure(bg=COLORS['bg_primary'])

        # Task storage (in-memory)
        self.tasks = []
        self.task_id_counter = 0

        # Configure ttk styles for modern look
        self.setup_styles()

        # Build the UI
        self.create_ui()

        # Bind events
        self.root.protocol("WM_DELETE_WINDOW", self.on_closing)

    def center_window(self):
        """Center the window on the screen."""
        self.root.update_idletasks()
        width = 900
        height = 550
        x = (self.root.winfo_screenwidth() // 2) - (width // 2)
        y = (self.root.winfo_screenheight() // 2) - (height // 2)
        self.root.geometry(f'{width}x{height}+{x}+{y}')

    def setup_styles(self):
        """Configure ttk styles for modern appearance."""
        style = ttk.Style()
        style.theme_use('clam')

        # Configure Treeview (task list) style
        style.configure(
            'Modern.Treeview',
            background=COLORS['bg_secondary'],
            foreground=COLORS['text_primary'],
            fieldbackground=COLORS['bg_secondary'],
            borderwidth=0,
            font=FONTS['body'],
            rowheight=45,
        )
        style.configure(
            'Modern.Treeview.Heading',
            background=COLORS['sidebar'],
            foreground=COLORS['text_primary'],
            font=FONTS['body_bold'],
```

```
            borderwidth=0,
        )
        style.map(
            'Modern.Treeview',
            background=[('selected', COLORS['accent'])],
        )

        # Configure Scrollbar style
        style.configure(
            'Modern.Vertical.TScrollbar',
            background=COLORS['bg_secondary'],
            troughcolor=COLORS['bg_primary'],
            borderwidth=0,
        )

    def create_ui(self):
        """Create the complete user interface."""
        # ==================== MAIN CONTAINER ====================
        main_container = tk.Frame(
            self.root,
            bg=COLORS['bg_primary']
        )
        main_container.pack(fill=tk.BOTH, expand=True)

        # ==================== SIDEBAR ====================
        self.create_sidebar(main_container)

        # ==================== MAIN CONTENT AREA ====================
        self.create_main_content(main_container)

    def create_sidebar(self, parent):
        """Create the sidebar with title and navigation."""
        sidebar = tk.Frame(
            parent,
            bg=COLORS['sidebar'],
            width=250
        )
        sidebar.pack(side=tk.LEFT, fill=tk.Y)
        sidebar.pack_propagate(False)

        # App Logo/Icon Area
        logo_frame = tk.Frame(sidebar, bg=COLORS['sidebar'],
height=100)
        logo_frame.pack(fill=tk.X, pady=(30, 10))
```

```
        # App Icon (circle with checkmark)
        icon_canvas = tk.Canvas(
            logo_frame,
            width=60,
            height=60,
            bg=COLORS['sidebar'],
            highlightthickness=0
        )
        icon_canvas.pack()
        icon_canvas.create_oval(5, 5, 55, 55, fill=COLORS['accent'],
outline='')
        icon_canvas.create_text(30, 30, text='✓', font=('Segoe UI', 24,
'bold'),
                                fill='white', tags='checkmark')

        # App Title
        title_label = tk.Label(
            sidebar,
            text="Smart Task\nManager",
            font=FONTS['title'],
            bg=COLORS['sidebar'],
            fg=COLORS['text_primary'],
            justify=tk.CENTER
        )
        title_label.pack(pady=(10, 5))

        # Subtitle
        subtitle_label = tk.Label(
            sidebar,
            text="Organize your work efficiently",
            font=FONTS['subtitle'],
            bg=COLORS['sidebar'],
            fg=COLORS['text_secondary']
        )
        subtitle_label.pack(pady=(0, 30))

        # Stats Section
        self.create_stats_section(sidebar)

        # Spacer
        tk.Frame(sidebar, bg=COLORS['sidebar']).pack(fill=tk.Y,
expand=True)

        # Footer
        footer_label = tk.Label(
```

```
            sidebar,
            text="v1.0 | College UI Lab",
            font=FONTS['small'],
            bg=COLORS['sidebar'],
            fg=COLORS['text_muted']
        )
        footer_label.pack(pady=(0, 20))

    def create_stats_section(self, parent):
        """Create statistics display in sidebar."""
        stats_frame = tk.Frame(
            parent,
            bg=COLORS['bg_secondary'],
            padx=20,
            pady=20
        )
        stats_frame.pack(fill=tk.X, padx=15, pady=(0, 20))

        # Stats Title
        stats_title = tk.Label(
            stats_frame,
            text="STATISTICS",
            font=FONTS['small'],
            bg=COLORS['bg_secondary'],
            fg=COLORS['text_muted']
        )
        stats_title.pack(anchor=tk.W, pady=(0, 15))

        # Total Tasks
        total_frame = tk.Frame(stats_frame, bg=COLORS['bg_secondary'])
        total_frame.pack(fill=tk.X, pady=5)
        tk.Label(
            total_frame,
            text="Total Tasks:",
            font=FONTS['body'],
            bg=COLORS['bg_secondary'],
            fg=COLORS['text_secondary']
        ).pack(side=tk.LEFT)
        self.total_label = tk.Label(
            total_frame,
            text="0",
            font=FONTS['body_bold'],
            bg=COLORS['bg_secondary'],
            fg=COLORS['text_primary']
        )
```

```
        self.total_label.pack(side=tk.RIGHT)

        # Completed Tasks
        completed_frame = tk.Frame(stats_frame,
bg=COLORS['bg_secondary'])
        completed_frame.pack(fill=tk.X, pady=5)
        tk.Label(
            completed_frame,
            text="Completed:",
            font=FONTS['body'],
            bg=COLORS['bg_secondary'],
            fg=COLORS['text_secondary']
        ).pack(side=tk.LEFT)
        self.completed_label = tk.Label(
            completed_frame,
            text="0",
            font=FONTS['body_bold'],
            bg=COLORS['bg_secondary'],
            fg=COLORS['success']
        )
        self.completed_label.pack(side=tk.RIGHT)

        # Pending Tasks
        pending_frame = tk.Frame(stats_frame,
bg=COLORS['bg_secondary'])
        pending_frame.pack(fill=tk.X, pady=5)
        tk.Label(
            pending_frame,
            text="Pending:",
            font=FONTS['body'],
            bg=COLORS['bg_secondary'],
            fg=COLORS['text_secondary']
        ).pack(side=tk.LEFT)
        self.pending_label = tk.Label(
            pending_frame,
            text="0",
            font=FONTS['body_bold'],
            bg=COLORS['bg_secondary'],
            fg=COLORS['pending']
        )
        self.pending_label.pack(side=tk.RIGHT)

    def create_main_content(self, parent):
        """Create the main content area with task list and controls."""
        # Main content frame
```

```
        content = tk.Frame(
            parent,
            bg=COLORS['bg_primary']
        )
        content.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=30,
pady=30)

        # ==================== HEADER SECTION ====================
        header = tk.Frame(content, bg=COLORS['bg_primary'])
        header.pack(fill=tk.X, pady=(0, 20))

        # Section Title
        section_title = tk.Label(
            header,
            text="My Tasks",
            font=FONTS['heading'],
            bg=COLORS['bg_primary'],
            fg=COLORS['text_primary']
        )
        section_title.pack(side=tk.LEFT)

        # ==================== INPUT SECTION ====================
        input_frame = tk.Frame(content, bg=COLORS['bg_primary'])
        input_frame.pack(fill=tk.X, pady=(0, 20))

        # Task Input Field
        self.task_entry = tk.Entry(
            input_frame,
            font=FONTS['body'],
            bg=COLORS['bg_secondary'],
            fg=COLORS['text_primary'],
            insertbackground=COLORS['text_primary'],
            relief=tk.FLAT,
            bd=0,
            highlightthickness=1,
            highlightcolor=COLORS['accent'],
            highlightbackground=COLORS['border']
        )
        self.task_entry.pack(side=tk.LEFT, fill=tk.X, expand=True,
ipady=12, ipadx=15)
        self.task_entry.insert(0, "Enter a new task...")
        self.task_entry.bind('<FocusIn>', lambda e:
self.on_entry_focus_in())
        self.task_entry.bind('<FocusOut>', lambda e:
self.on_entry_focus_out())
```

```
        # Add Button
        add_button = tk.Button(
            input_frame,
            text="+ Add Task",
            font=FONTS['button'],
            bg=COLORS['accent'],
            fg=COLORS['text_primary'],
            activebackground=COLORS['accent_hover'],
            activeforeground=COLORS['text_primary'],
            relief=tk.FLAT,
            bd=0,
            cursor='hand2',
            padx=20,
            pady=12,
            command=self.add_task
        )
        add_button.pack(side=tk.LEFT, padx=(15, 0))

        # Bind Enter key to add task
        self.task_entry.bind('<Return>', lambda e: self.add_task())

        # ==================== TASK LIST SECTION ====================
        list_frame = tk.Frame(content, bg=COLORS['bg_primary'])
        list_frame.pack(fill=tk.BOTH, expand=True)

        # Treeview for tasks
        columns = ('id', 'task', 'status')
        self.tree = ttk.Treeview(
            list_frame,
            columns=columns,
            show='tree headings',
            style='Modern.Treeview',
            selectmode='browse'
        )

        # Configure columns
        self.tree.heading('#0', text='')
        self.tree.heading('id', text='ID')
        self.tree.heading('task', text='Task')
        self.tree.heading('status', text='Status')

        self.tree.column('#0', width=0, stretch=tk.NO)
        self.tree.column('id', width=60, minwidth=60, anchor=tk.CENTER)
        self.tree.column('task', width=400, minwidth=300)
```

```
        self.tree.column('status', width=120, minwidth=100,
anchor=tk.CENTER)

        # Scrollbar
        scrollbar = ttk.Scrollbar(
            list_frame,
            orient=tk.VERTICAL,
            command=self.tree.yview,
            style='Modern.Vertical.TScrollbar'
        )
        self.tree.configure(yscrollcommand=scrollbar.set)

        # Pack tree and scrollbar
        self.tree.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

        # Bind selection event
        self.tree.bind('<<TreeviewSelect>>', self.on_task_select)

        # ==================== ACTION BUTTONS SECTION
====================
        actions_frame = tk.Frame(content, bg=COLORS['bg_primary'])
        actions_frame.pack(fill=tk.X, pady=(20, 0))

        # Complete Task Button
        complete_btn = tk.Button(
            actions_frame,
            text="✓ Mark Complete",
            font=FONTS['button'],
            bg=COLORS['success'],
            fg=COLORS['text_primary'],
            activebackground=COLORS['success_bg'],
            activeforeground=COLORS['text_primary'],
            relief=tk.FLAT,
            bd=0,
            cursor='hand2',
            padx=25,
            pady=12,
            command=self.mark_complete
        )
        complete_btn.pack(side=tk.LEFT, padx=(0, 10))

        # Delete Task Button
        delete_btn = tk.Button(
            actions_frame,
```

```
            text="✗ Delete Task",
            font=FONTS['button'],
            bg=COLORS['error'],
            fg=COLORS['text_primary'],
            activebackground='#B91C1C',
            activeforeground=COLORS['text_primary'],
            relief=tk.FLAT,
            bd=0,
            cursor='hand2',
            padx=25,
            pady=12,
            command=self.delete_task
        )
        delete_btn.pack(side=tk.LEFT)

        # Selected task label
        self.selected_label = tk.Label(
            actions_frame,
            text="No task selected",
            font=FONTS['small'],
            bg=COLORS['bg_primary'],
            fg=COLORS['text_muted']
        )
        self.selected_label.pack(side=tk.RIGHT, pady=(0, 0))

    # ==================== EVENT HANDLERS ====================

    def on_entry_focus_in(self):
        """Handle entry focus in event."""
        if self.task_entry.get() == "Enter a new task...":
            self.task_entry.delete(0, tk.END)
            self.task_entry.configure(
                fg=COLORS['text_primary'],
                highlightcolor=COLORS['accent'],
                highlightbackground=COLORS['border']
            )

    def on_entry_focus_out(self):
        """Handle entry focus out event."""
        if not self.task_entry.get():
            self.task_entry.insert(0, "Enter a new task...")
            self.task_entry.configure(fg=COLORS['text_muted'])

    def on_task_select(self, event):
        """Handle task selection event."""
```

```
        selection = self.tree.selection()
        if selection:
            item = self.tree.item(selection[0])
            task_id = item['values'][0]
            task_text = item['values'][1]
            self.selected_label.config(text=f"Selected:
{task_text[:30]}...")
        else:
            self.selected_label.config(text="No task selected")

    def on_closing(self):
        """Handle window closing event."""
        if messagebox.askokcancel("Exit", "Are you sure you want to
exit?"):
            self.root.destroy()
            sys.exit()

    # ==================== TASK OPERATIONS ====================

    def add_task(self):
        """Add a new task to the list."""
        task_text = self.task_entry.get().strip()

        # Input validation
        if not task_text or task_text == "Enter a new task...":
            self.show_error("Please enter a task description.")
            self.task_entry.focus()
            return

        # Create task
        self.task_id_counter += 1
        task = {
            'id': self.task_id_counter,
            'text': task_text,
            'status': 'Pending'
        }
        self.tasks.append(task)

        # Insert into treeview
        self.tree.insert(
            '',
            'end',
            values=(task['id'], task['text'], task['status']),
            tags=('pending',)
        )
```

```
        # Clear entry
        self.task_entry.delete(0, tk.END)
        self.task_entry.insert(0, "Enter a new task...")
        self.task_entry.configure(fg=COLORS['text_muted'])

        # Update stats
        self.update_stats()

        # Show success message
        self.show_success("Task added successfully!")

        # Select the newly added task
        last_item = self.tree.get_children()[-1]
        self.tree.selection_set(last_item)
        self.tree.focus(last_item)

    def mark_complete(self):
        """Mark selected task as completed."""
        selection = self.tree.selection()
        if not selection:
            self.show_error("Please select a task to mark as
complete.")
            return

        item = selection[0]
        item_values = self.tree.item(item)['values']

        # Check if already completed
        if item_values[2] == 'Completed':
            self.show_error("This task is already completed!")
            return

        # Update task in memory
        task_id = item_values[0]
        for task in self.tasks:
            if task['id'] == task_id:
                task['status'] = 'Completed'
                break

        # Update treeview
        self.tree.item(item, values=(item_values[0], item_values[1],
'Completed'))

        # Update stats
```

```
        self.update_stats()

        # Show success
        self.show_success("Task marked as completed! ✓")

    def delete_task(self):
        """Delete selected task."""
        selection = self.tree.selection()
        if not selection:
            self.show_error("Please select a task to delete.")
            return

        item = selection[0]
        item_values = self.tree.item(item)['values']

        # Confirm deletion
        if not messagebox.askyesno("Confirm Delete", f"Delete task:
'{item_values[1]}'?"):
            return

        # Remove from memory
        task_id = item_values[0]
        self.tasks = [task for task in self.tasks if task['id'] !=
task_id]

        # Remove from treeview
        self.tree.delete(item)

        # Update stats
        self.update_stats()

        # Update selection label
        self.selected_label.config(text="No task selected")

        # Show success
        self.show_success("Task deleted successfully!")

    def update_stats(self):
        """Update the statistics display."""
        total = len(self.tasks)
        completed = sum(1 for task in self.tasks if task['status'] ==
'Completed')
        pending = total - completed

        self.total_label.config(text=str(total))
```

```python
        self.completed_label.config(text=str(completed))
        self.pending_label.config(text=str(pending))

    def show_success(self, message):
        """Show success message popup."""
        messagebox.showinfo("Success", message)

    def show_error(self, message):
        """Show error message popup."""
        messagebox.showerror("Error", message)


#
==========================================================================
======
# MAIN ENTRY POINT
#
==========================================================================
======
def main():
    """Main function to run the application."""
    root = tk.Tk()
    app = SmartTaskManager(root)
    root.mainloop()


if __name__ == "__main__":
    main()
```

OUTPUT:

```
PS C:\Users\LENOVO\Desktop\UID EXPERIMENT 3> python vui_create_file.py
 Say the file name to create
❌ Could not understand voice
PS C:\Users\LENOVO\Desktop\UID EXPERIMENT 3> python vui_create_file.py
 Say the file name to create
File created: testfile.txt
✅ File created successfully!
```

4. USER SATISFACTION COMPARISON:

PYTHON CODE:

```
print("User Satisfaction Survey (1 to 5)")

cli = int(input("Rate CLI file creation: "))
gui = int(input("Rate GUI file creation: "))
vui = int(input("Rate VUI file creation: "))

avg = (cli + gui + vui) / 3

print("\nResults")
print("CLI:", cli)
print("GUI:", gui)
print("VUI:", vui)
print("Average Satisfaction:", avg)
```

OUTPUT:

```
PS C:\Users\LENOVO\Desktop\UID EXPERIMENT 3> python survey.py
User Satisfaction Survey (1 to 5)
Rate CLI file creation: 4
Rate GUI file creation: 4
Rate VUI file creation: 5

Results
CLI: 4
GUI: 4
VUI: 5
Average Satisfaction: 4.333333333333333
PS C:\Users\LENOVO\Desktop\UID EXPERIMENT 3>
```